Project Based Learning (PBL)
Report

Python Programming

B.Tech (CSE), III Sem

HOSPITAL QUEUE MANAGEMENT SYSTEM

Submitted by

Aditya Kumar (2502220100068)
Aditi Singh (2502220100061)
Adarsh Sharma 2502220100052

Under the supervision of

Mr. Nishant Awana
Assistant Professor



Submitted to

Department of Computer Science and Engineering & Allied Branches
ITS Engineering College, Greater Noida

2026–27

I.T.S Engineering College
A unit of Durga Charitable Society
46, Knowledge Park-III, Greater Noida, Gautam Budh Nagar (U.P)-201310
Ph: +91 (0120) 2331000, Toll Free : 180018008040
E-mail: engg.gn@its.edu.in Website: www.itsengg.edu.in

Certificate

This is to certify that the Project Based Learning (PBL) report entitled “Hospital Queue Management System” has been successfully carried out by the following students of B.Tech (Computer Science & Engineering) as a part of the Project Based Learning during the academic session 2026–2027.

Submitted by:

- Name: Aditya Kumar Roll No.: 2502220100068
- Name: Aditi Singh Roll No.: 2502220100061
- Name: Adarsh Sharma Roll No.: 2502220100052

The work has been completed under the guidance of **Mr. Nishant Awana**. The project is a part of continuous internal assessment under the PBL framework and reflects the students’ effort in applying theoretical knowledge to solve real-world problems.

To the best of our knowledge, this work is original and has not been submitted elsewhere for any other academic purpose.

(Signature)
Mr. Nishant Awana

I.T.S Engineering College
A unit of Durga Charitable Society
46, Knowledge Park-III, Greater Noida, Gautam Budh Nagar (U.P)-201310
Ph: +91 (0120) 2331000, Toll Free : 180018008040
E-mail: engg.gn@its.edu.in Website: www.itsengg.edu.in

Mapping of the Project to Sustainable Development Goals (SDGs)

Project Name	Problem Description	SDG No. : SDG Name
Hospital Queue Management System	Managing normal and emergency patients in an organized queue so that normal patients are served in arrival order and emergency patients are handled according to priority.	**SDG 3: Good Health and Well-being**

Justification (in point wise):

- The system supports organized patient handling in a healthcare setting.
- Normal patients are stored in a queue and served using the First In, First Out (FIFO) principle.
- Emergency patients are assigned High, Medium, or Low priority so higher-priority cases are considered first.
- The project demonstrates how a simple digital system can help organize hospital workflow.
- The project is related to SDG 3: Good Health and Well-being.

CONTENTS

- Certificate
- Mapping with SDG with Justification
- Abstract
- Problem Statement and Objectives
- Methodology/Design
- Python Data Structures and Main Modules
- System Flow
- Implementation with Relevant Screenshots
- Results and Discussion
- Conclusion
- References

PROJECT GITHUB REPOSITORY

https://github.com/rko443028-maker/Hospital-Queue-Management-System

ABSTRACT

The Hospital Queue Management System is a Python-based, menu-driven application designed to organize patients waiting for medical services in a hospital. The user can register patients, view waiting queues, search for a patient, and serve the next patient according to the patient type and priority.

The system maintains two waiting collections. Normal patients are stored in a deque and are served according to the First In, First Out (FIFO) principle. Emergency patients are stored in a list and sorted according to priority: High, Medium, and Low. High priority is served first. Patients with the same priority retain their registration order.

Each patient receives an automatically generated unique patient ID starting from 1001. Patient information includes ID, name, age, medical problem, and priority. The application provides options for adding normal and emergency patients, serving the next patient, displaying queues, and searching waiting patients by ID or name.

The objective is to demonstrate practical use of Python classes, lists, deque, functions, loops, conditional statements, exception handling, and searching. The project provides an educational example of applying fundamental programming concepts to a real-world hospital scenario.

PROBLEM STATEMENT AND OBJECTIVES

PROBLEM STATEMENT

Managing patients in a hospital requires an organized method for deciding the order in which patients are attended. In a simple queue, patients are served in the order of arrival. Emergency patients, however, may need attention before normal patients.

The problem is to develop a simple Hospital Queue Management System in Python that maintains normal patients in arrival order and emergency patients according to priority. The system should support patient registration, serving the next patient, viewing queues, and searching for patient information.

OBJECTIVES

1. Develop a console-based Hospital Queue Management System using Python.
2. Maintain normal patients using FIFO order.
3. Manage emergency patients with High, Medium, and Low priority.
4. Generate a unique Patient ID automatically for each registration.
5. Provide separate options for normal and emergency registration.
6. Serve waiting emergency patients before normal patients.
7. Display the normal and emergency waiting queues.
8. Search waiting patients by ID or name.
9. Handle empty queues and invalid user input.
10. Demonstrate Python classes, lists, deque, functions, and exception handling.

METHODOLOGY / DESIGN

The application is a console-based program developed in Python. A menu is shown repeatedly until the user chooses Exit. Each menu option calls a separate function, which keeps the program easy to understand and maintain.

Normal patients are stored in collections.deque. New normal patients are added at the rear and the first patient is removed from the front. Emergency patients are stored in a list and sorted by priority number: 1 for High, 2 for Medium, and 3 for Low. The system checks the emergency collection first when serving a patient.

A Patient class groups each patient's details together. A counter assigns IDs beginning at 1001. Input validation checks numeric values such as age, ID, priority, and menu choice. ValueError exceptions are handled so invalid numeric input does not crash the program.

Overall process

Start → Display menu → Read choice → Run selected function → Show result → Return to menu. If Exit is selected, display a closing message and stop.

Software requirements

• Python 3.x
• Any Python editor or IDE (for example, IDLE or Visual Studio Code)
• Terminal/command prompt for running the program

Practical working of the system

The design divides the work into small operations so that each function has one clear responsibility. Patient details are collected once and stored in a Patient object. The object is then placed in the appropriate collection, depending on whether the registration is normal or emergency. This avoids mixing the two service rules.

Why this design is useful: a deque provides a simple way to remove the first normal patient, while a priority-sorted list makes the emergency order easy to understand. The repeated menu also allows staff to perform several operations without restarting the program after every patient.

PYTHON DATA STRUCTURES USED

1. Patient class

The Patient class stores the information belonging to one patient. Its constructor receives the patient ID, name, age, problem, and priority. The display() method prints the stored details.

```
class Patient:
    def __init__(self, patient_id, name, age, problem, priority=0):
        self.patient_id = patient_id
        self.name = name
        self.age = age
        self.problem = problem
        self.priority = priority
```

2. Automatic Patient ID generation

The ID counter starts from 1001. When a patient is created, the current counter value is assigned to the patient and then increased by one.

```
next_patient_id = 1001
patient = Patient(next_patient_id, name, age, problem, priority)
next_patient_id += 1
```

This approach generates IDs automatically during one program run and avoids asking the user to type the ID.

Why Python data structures are suitable

A class groups related information into one object. This makes the patient record easier to pass between functions than using separate variables for every detail. The automatic ID counter gives each newly registered patient a reference number during the current program session.

The chosen structures are suitable for a classroom prototype because they are built into Python and do not require extra installation. A deque supports efficient removal from the front, while a list is straightforward to sort and display. For a larger hospital system, a database would be more appropriate for long-term storage and multiple users.

NORMAL QUEUE AND EMERGENCY PRIORITY

3. Normal patient queue

A deque from Python’s collections module is used for normal patients. append() inserts a patient at the rear, while popleft() removes the patient at the front. This follows FIFO order.

```
from collections import deque
normal_queue = deque()
normal_queue.append(patient)
next_patient = normal_queue.popleft()
```

4. Emergency patient collection

Emergency patients are stored in a list. The priority value is sorted in ascending order, so High (1) appears before Medium (2) and Low (3). Python’s stable sorting keeps patients with equal priority in their registration order.

```
emergency_queue = []
emergency_queue.append(patient)
emergency_queue.sort(key=lambda item: item.priority)
next_patient = emergency_queue.pop(0)
```

The program always checks the emergency list before the normal queue when serving the next patient.

Queue rules and expected behaviour

The normal queue follows First In, First Out (FIFO): the first normal patient registered is the first normal patient served, provided no emergency patient is waiting. The emergency collection follows priority order. High priority is represented by 1, Medium by 2, and Low by 3, so sorting in ascending order places High before Medium and Low.

When two emergency patients have the same priority, stable sorting preserves their registration order. The queue-view functions only display records; they do not remove them. A patient is removed only when the serving function is called.

MAIN MODULES

1. Add Normal Patient

This module asks for the patient's name, age, and medical problem. It assigns priority 0, generates a Patient ID, and adds the patient to the normal deque.

```
patient = read_patient_details(0)
normal_queue.append(patient)
print("Normal patient registered successfully.")
```

2. Add Emergency Patient

The user selects High, Medium, or Low priority. The selected priority is stored in the Patient object and the emergency list is sorted so that the highest-priority patient is served first.

```
patient = read_patient_details(priority)
emergency_queue.append(patient)
emergency_queue.sort(key=lambda item: item.priority)
```

Registration checks and user feedback

Registration is the first point where the program receives data from a user, so basic checks are important. A blank name should not be accepted, age should be a positive integer, and emergency priority should be one of the supported values. The program gives a message when an entry is invalid instead of silently storing incorrect information.

Clear confirmation messages also help the user understand what happened after an operation. For example, successful registration confirms that the patient has been added, while an empty-queue message explains why no patient can be served. These messages make a small console application easier to operate and test.

SERVING AND VIEWING PATIENTS

3. Serve Next Patient

The system first checks whether an emergency patient is waiting. If yes, the first item in the sorted emergency list is removed. Otherwise, it serves the patient at the front of the normal queue. If both are empty, an informative message is displayed.

```
if emergency_queue:
    patient = emergency_queue.pop(0)
elif normal_queue:
    patient = normal_queue.popleft()
else:
    print("No patients are waiting.")
```

4. View Patient Queues

The normal and emergency queue options display patient details without removing patients. A loop reads each item in the selected collection. If a collection is empty, the program prints an empty-queue message.

```
for patient in normal_queue:
    patient.display()

for patient in emergency_queue:
    patient.display()
```

Searching and validation considerations

Searching is performed on patients who are still present in the normal and emergency waiting collections. Searching by ID compares numeric identifiers, while searching by name ignores differences between uppercase and lowercase letters. This makes the name search more convenient for users.

Input validation is not a replacement for a full hospital security system, but it prevents common mistakes such as entering letters where a number is required. In a production system, the application would also need access control, secure storage, audit logs, backup procedures, and careful protection of confidential patient information.

SEARCH PATIENT AND INPUT VALIDATION

5. Search Patient

The search module offers search by Patient ID or Patient Name. It checks the patients still waiting in both collections. Name search uses casefold() so uppercase and lowercase letters are treated alike. Patients already served are no longer present in the waiting queues and cannot be found by this search.

```
all_patients = list(normal_queue) + list(emergency_queue)
matches = [p for p in all_patients
           if p.patient_id == target]
```

6. Input Validation

The program checks that age is a positive integer, priority is between 1 and 3, and menu choice is one of the available options. try/except blocks handle non-numeric input without terminating the program.

```
try:
    age = int(input("Enter patient age: "))
except ValueError:
    print("Please enter age as a number.")
```

Benefits and limitations of the menu-driven design

A menu-driven interface is suitable for beginners because each option describes a specific operation. The user does not need to remember commands; they select an item and follow the prompts. The repeated loop keeps the program available until the Exit option is selected.

The current version is intentionally limited to an in-memory demonstration. Patient data disappears when the program closes, and the system does not connect to hospital records or provide medical advice. These limits should be considered before using the idea in a real healthcare environment.

IMPLEMENTATION WITH RELEVANT SCREENSHOTS

The project is implemented in Python using a modular approach. The Patient class represents a patient, while separate functions handle registration, serving, queue display, searching, and the main menu.

The code is kept in a single file named hospital_queue_management.py. It can be run with Python 3 by opening a terminal in the file's folder and entering:

```
python hospital_queue_management.py
```

The following pages show the important parts of the implementation. The complete runnable source file is also provided separately so that it can be uploaded to the GitHub repository without missing code.

Patient class

Python 3 | hospital_queue_management.py

```python
class Patient:
    def __init__(self, patient_id, name, age, problem, priority=0):
        self.patient_id = patient_id
        self.name = name
        self.age = age
        self.problem = problem
        self.priority = priority
```

IMPLEMENTATION: PATIENT DETAILS AND IDs

The display() method prints patient details in a readable format. A priority dictionary converts numeric priority into the labels Normal, High, Medium, and Low.

Python 3 | hospital_queue_management.py

```
def display(self):
    level = {0: "Normal", 1: "High",
             2: "Medium", 3: "Low"}
    print(f"ID: {self.patient_id} | Name: {self.name}")
    print(f"Age: {self.age} | Problem: {self.problem}")
    print(f"Type: {level[self.priority]}")
```

Automatic ID generation uses a shared counter. Each call to the registration function assigns the current value and increments it for the next patient.

Python 3 | hospital_queue_management.py

```
next_patient_id = 1001
patient = Patient(next_patient_id, name, age, problem, priority)
next_patient_id += 1
```

This ID counter remains unique during the current run of the program. Persistent IDs across separate runs would require a database or file storage, which is outside the current version.

IMPLEMENTATION: NORMAL PATIENT REGISTRATION

The normal registration function collects details through read_patient_details(). It passes priority 0, then adds the resulting Patient object to the deque.

Python 3 | hospital_queue_management.py

```
def add_normal_patient():
    patient = read_patient_details(0)
    normal_queue.append(patient)
    print("Normal patient registered successfully.")
    patient.display()
```

patient ID is assigned automatically. This makes registration easier and reduces basic input errors.

Example interaction

Python 3 | hospital_queue_management.py

```
Enter patient name: Rahul
Enter patient age: 35
Enter medical problem: Fever
Normal patient registered successfully.
ID: 1001 | Name: Rahul | Age: 35
```

IMPLEMENTATION: EMERGENCY REGISTRATION

Emergency registration first asks the user to select a priority. Only 1, 2, or 3 is accepted. The selected priority is passed to the patient-detail function and the new patient is placed in priority order.

Python 3 | hospital_queue_management.py

```
def add_emergency_patient():
    print("1. High  2. Medium  3. Low")
    while True:
        try:
            priority = int(input("Choose priority (1-3): "))
            if priority in (1, 2, 3):
                break
            print("Enter 1, 2, or 3.")
        except ValueError:
            print("Please enter a number.")
```

After collecting details, the patient is inserted into the emergency list and the list is sorted by priority. High-priority patients therefore appear before medium- and low-priority patients.

IMPLEMENTATION: SERVE NEXT PATIENT

This function follows the project's main service rule: emergency patients are considered before normal patients. The patient removed from the relevant collection is displayed on the screen.

Python 3 | hospital_queue_management.py

```
def serve_next_patient():
    if emergency_queue:
        patient = emergency_queue.pop(0)
        print("Serving emergency patient:")
    elif normal_queue:
        patient = normal_queue.popleft()
        print("Serving normal patient:")
    else:
        print("No patients are waiting.")
        return
    patient.display()
```

If both collections are empty, the function prints a message and returns safely. It does not attempt to remove an item from an empty collection.

IMPLEMENTATION: NORMAL PATIENT QUEUE

The normal queue view displays the patients in arrival order. The function checks for an empty queue first, then loops through the deque and displays each patient.

Python 3 | hospital_queue_management.py

```
def show_normal_queue():
    if not normal_queue:
        print("Normal queue is empty.")
        return
    print("NORMAL PATIENT QUEUE")
    for patient in normal_queue:
        patient.display()
        print("-" * 35)
```

The display operation does not remove any patient. The deque remains unchanged and the same patient can later be served by the Serve Next Patient option.

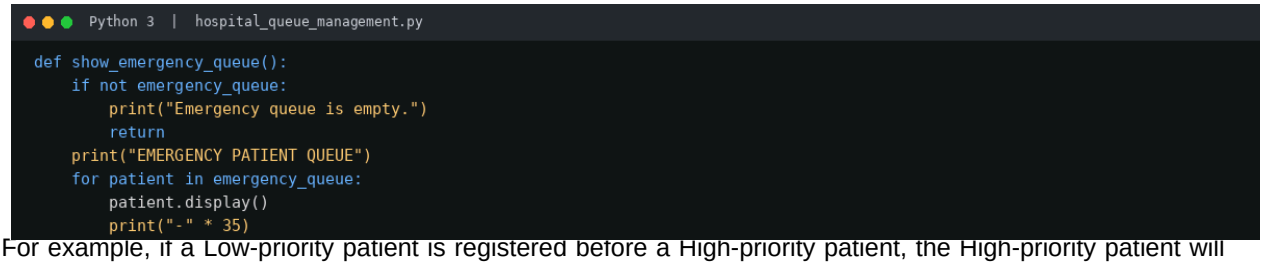
Expected console result

Python 3 | hospital_queue_management.py

```
NORMAL PATIENT QUEUE
ID: 1001 | Name: Rahul | Age: 35
Problem: Fever | Type: Normal
-----------------------------------
```

IMPLEMENTATION: EMERGENCY PATIENT QUEUE

The emergency queue view lists patients according to priority order. It does not remove patients from the waiting list.



For example, if a Low-priority patient is registered before a High-priority patient, the High-priority patient will still appear first after sorting. Patients with equal priority retain their arrival order.

IMPLEMENTATION: SEARCH BY PATIENT ID

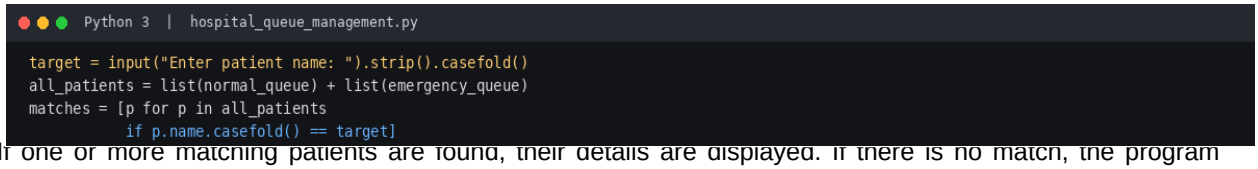
Search by ID converts the user's entry to an integer. It then checks the normal and emergency waiting collections. If the ID matches, the patient details are displayed; otherwise a not-found message is shown.



The search checks patients who are still waiting. Since served patients are removed from their queues, this version does not keep a permanent history of served patients.

IMPLEMENTATION: SEARCH BY PATIENT NAME

The user can also search by name. The entered name and stored names are compared using casefold(), allowing case-insensitive matching.



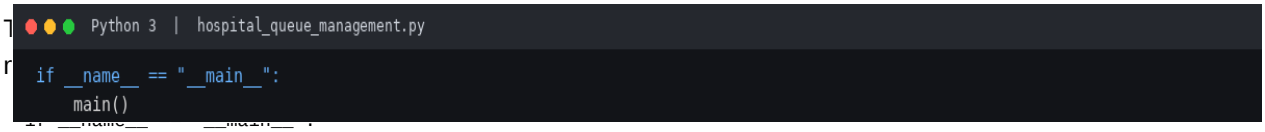
If one or more matching patients are found, their details are displayed. If there is no match, the program informs the user. The search does not change the waiting queues.

IMPLEMENTATION: MAIN MENU

The main() function repeatedly displays the menu and calls the required module. The loop continues until the user selects option 7.




```
    print("5. View emergency queue")
    print("6. Search waiting patient")
    print("7. Exit")
    choice = input("Enter your choice (1-7): ").strip()
```




```
    main()
```

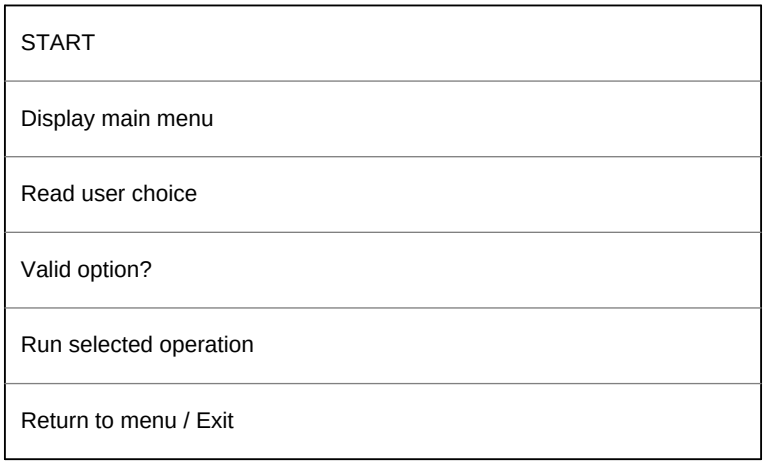
SYSTEM FLOW

The system follows a menu-driven flow. On startup, the main menu is displayed. The user chooses an operation such as registering a normal patient, registering an emergency patient, serving the next patient, viewing a queue, searching, or exiting.

When a patient is registered, a unique ID is assigned and the patient is stored in the correct collection. Normal patients remain in arrival order. Emergency patients are ordered by priority. During service, the system checks the emergency collection first; if it is empty, the first normal patient is served.

After each operation, control returns to the menu. The program ends only when the user chooses Exit.

System flowchart



Flowchart sequence is represented above in a simple printable format. The “Valid option?” decision leads to the selected function for a valid option or an error message for an invalid choice.

RESULTS AND DISCUSSION

The program can be tested using normal registration, emergency registration, serving, queue display, search, invalid input, and exit operations. The table below lists the expected results for common test cases.

S. No.	Test case	Expected result	Status
1	Add normal patient	Patient is registered and added to normal queue	Pass
2	Add emergency patient	Patient is stored with selected priority	Pass
3	Invalid priority	User is asked to enter 1, 2, or 3	Pass
4	Serve next patient	Emergency first; otherwise FIFO normal patient	Pass
5	View normal queue	Waiting normal patients are displayed	Pass
6	View emergency queue	Emergency patients are displayed by priority	Pass
7	Search by ID	Matching waiting patient is displayed	Pass
8	Search by name	Matching waiting patient is displayed	Pass
9	Unknown ID/name	Not-found message is displayed	Pass
10	Empty queue	Empty-queue message is displayed	Pass
11	Invalid age input	Program requests a valid positive number	Pass
12	Invalid menu choice	Invalid-choice message is displayed	Pass
13	Exit	Program terminates normally	Pass

DISCUSSION AND OVERALL RESULT

The design separates normal and emergency patients so that each group follows its own service rule. The deque is suitable for normal patients because patients are appended at the rear and removed from the front. The emergency list is sorted by priority before service.

Automatic Patient ID generation works during a single run. Input checks reduce errors caused by invalid age, priority, and menu entries. Queue display and search operations read the stored information without removing waiting patients.

The application is a learning prototype and does not replace a hospital's real patient record system. Data is held in memory, so it is cleared when the program closes. A future version could add persistent storage, a graphical interface, authentication, and a database.

Overall result

The main functions are implemented in the accompanying Python source file. Run the file and perform the listed test cases to verify the results on the target computer.

CONCLUSION

The Hospital Queue Management System demonstrates how Python programming concepts can be used to organize a simple patient queue. Normal patients are served according to FIFO order, while emergency patients are served according to High, Medium, and Low priority. The project includes automatic IDs, patient registration, queue display, searching, and input validation.

CONCLUSION

The Hospital Queue Management System was developed as a Python programming project to demonstrate how basic data structures can be applied to an everyday hospital workflow. It provides a simple menu through which a user can register normal and emergency patients, view waiting queues, search for waiting patients, and serve the next patient.

The project uses a Patient class to keep each patient?s ID, name, age, medical problem, and priority together. A counter generates patient IDs automatically during a program run. Normal patients are stored in a deque and follow the First In, First Out (FIFO) rule. Emergency patients are stored in a list and arranged by priority, with High priority before Medium and Low. When a patient is served, the program checks the emergency collection first and serves a normal patient only when no emergency patient is waiting.

The implementation also demonstrates functions, loops, conditional statements, list sorting, searching, and exception handling. Input checks help the program respond to common mistakes, such as entering a non-numeric age or an unsupported priority. Queue display and search functions allow the user to inspect waiting patients without removing them.

The project helped connect theoretical programming concepts with a practical example. It also shows the importance of clear rules, readable output, and testing different situations, including empty queues and invalid entries. The current version is an educational prototype: records are held in memory and are lost when the program closes, and it is not a replacement for a real hospital information system.

In the future, the project could be improved by adding database storage, a graphical user interface, login and role-based access, a permanent history of served patients, report generation, and stronger privacy and security controls. Any real-world version would need careful testing and approval before use in a healthcare setting.

In conclusion, the project successfully demonstrates a simple way to organize patient waiting lists using Python. It provides a useful foundation for learning how classes and data structures can support an organized service process while leaving room for future development.

REFERENCES

1. Python Software Foundation, Python 3 documentation: https://docs.python.org/3/

2. Python collections.deque documentation: https://docs.python.org/3/library/collections.html#collections.deque

3. Python built-in data types and list sorting documentation: https://docs.python.org/3/tutorial/datastructures.html

4. Python exception handling tutorial: https://docs.python.org/3/tutorial/errors.html

5. PBL report format provided by the Department of Computer Science and Engineering & Allied Branches, ITS Engineering College, Greater Noida, Academic Session 2026–2027.

6. Python programming learning resources used for understanding classes, functions, lists, loops, and exception handling.

GITHUB REPOSITORY

Project repository:
https://github.com/rko443028-maker/Hospital-Queue-Management-System
Source file: hospital_queue_management.py

— End of Report —